Student Name: -Kumbhar Dilip

College Name: - Indus University

Internship: - Cyber Security Internship

Organization: - DG Intern Hub

Week 7 Assignment: - VAPT (WEB APPLICATION TESTING WITH BURP SUITE)

SID: - dg/September/cyber/264

Mail: - dilipkumbhar.dginternshub@gmail.com

Task 7: - Theory + Practical

Sr. No.	Report Contents	Page No.
1	Cover Page	1
2	Table of Contents	2
3	Introduction to VAPT	3
4	Objectives of the Project	3
5	Tools and Technologies Used	4
6	Lab Setup and Target Application	5
7	Task 22: Burp Suite Setup and Interception	6
8	Task 23: Reconnaissance and Web Application Mapping	7
9	Task 24: Vulnerability Testing Using Burp Suite	8–9
10	Task 25: Directory and Endpoint Discovery	10
11	Task 26: Vulnerability Assessment Findings	11–13
12	Security Recommendations and Remediation	14
13	Conclusion and Learning Outcomes	15
14	References	15

1. Introduction and Objectives

Introduction to VAPT

Vulnerability Assessment and Penetration Testing (VAPT) is a security testing process used to identify and evaluate weaknesses in web applications, networks, and computer systems. It helps organizations understand security risks and take appropriate steps to protect their applications from cyberattacks.

In this project, I used Burp Suite Community Edition to perform basic web application security testing in an authorized practice environment. Burp Suite is a popular security testing tool that allows testers to intercept, inspect, and modify HTTP/HTTPS requests and responses between a browser and a web server. It helps analyze application behaviour, test user inputs, and identify potential vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), authentication issues, and parameter tampering.

This practical task helped me understand how web applications communicate with servers, how common vulnerabilities can be tested, and how security findings can be documented with proper evidence and remediation recommendations.

Objectives

The main objectives of this project are:

1. **Understand Web Application Testing:** Learn the basic concepts of web application security testing and the importance of identifying vulnerabilities.

2. **Intercept and Analyze Requests:** Use Burp Suite to capture and inspect HTTP/HTTPS requests and responses between the browser and the web server.

3. **Identify Common Vulnerabilities:** Perform authorized tests for vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), authentication weaknesses, and parameter tampering.

4. **Explore Web Application Endpoints:** Use Burp Suite to map website pages, input fields, parameters, and endpoints to understand the application's attack surface.

5. **Document Security Findings:** Prepare a professional VAPT report containing vulnerability descriptions, steps to reproduce, screenshots, potential impacts, and remediation methods.

6. **Improve Security Awareness:** Understand how vulnerabilities can affect web applications and learn practical methods to reduce security risks.

2. Tools, Lab Setup and Task 22 – Burp Suite Setup & Interception

1. Tools Used

- Burp Suite Community Edition: To intercept and analyze web requests.
- Mozilla Firefox: To browse the practice website.
- FoxyProxy Extension: To route Firefox traffic through Burp Suite.
- Practice Target: DVWA or OWASP Juice Shop in an authorized lab.

2. Lab Setup

1. Install and open Burp Suite Community Edition.
2. Start a temporary project and use the default settings.
3. Open Proxy → Proxy settings and verify that the proxy listener is running on 127.0.0.1:8080.
4. Install the FoxyProxy extension in Firefox.
5. Add a new proxy profile in FoxyProxy with these settings:

- Proxy Type: HTTP
- Proxy IP: 127.0.0.1
- Port: 8080

6. Save the profile and enable it in Firefox.

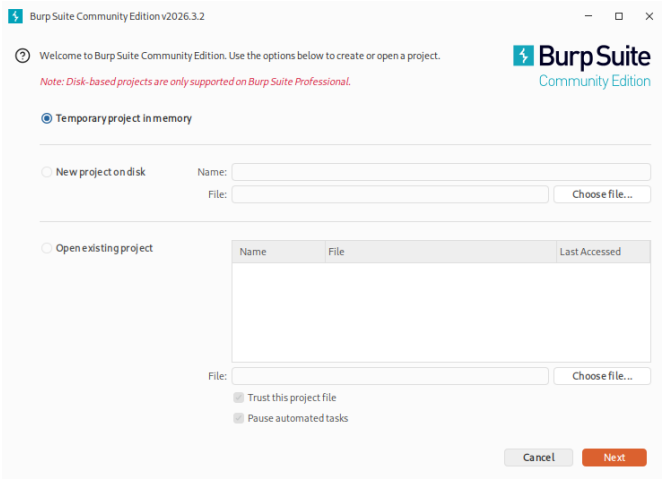
3. HTTPS Certificate Setup

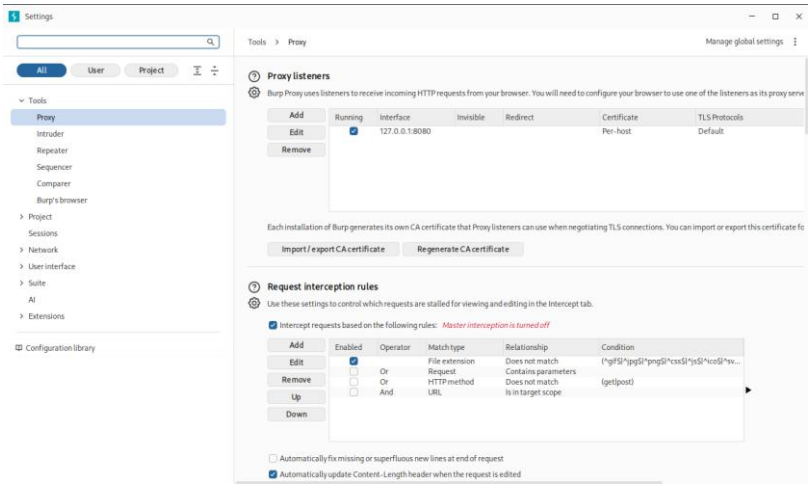
1. Enable the Burp proxy profile in Firefox.
2. Visit http://burp through the configured browser.
3. Download the Burp CA certificate.
4. In Firefox, open Settings → Privacy & Security → Certificates → View Certificates → Authorities.
5. Import the certificate and trust it for identifying websites.
6. Open your authorized HTTPS practice target and check whether it loads without certificate warnings.

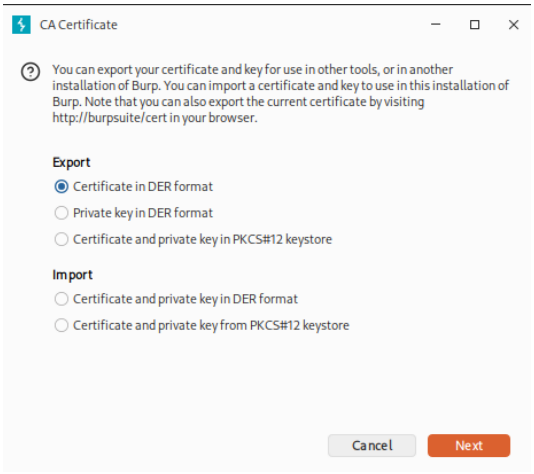
4. Intercepting HTTP/HTTPS Requests

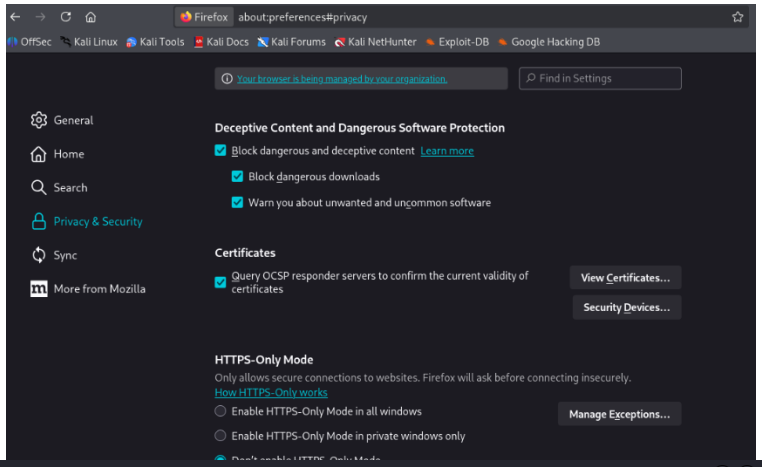
1. Open Burp Suite and select Proxy → Intercept.
2. Turn interception on.
3. Browse the practice website in Firefox.
4. Burp Suite will capture the request.
5. Inspect the request method, URL, headers, and parameters.
6. Click Forward to send the request to the server.

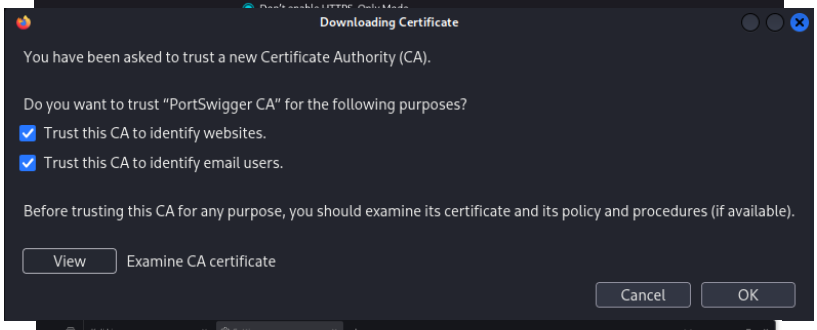
7. Review the server response and record your observations.

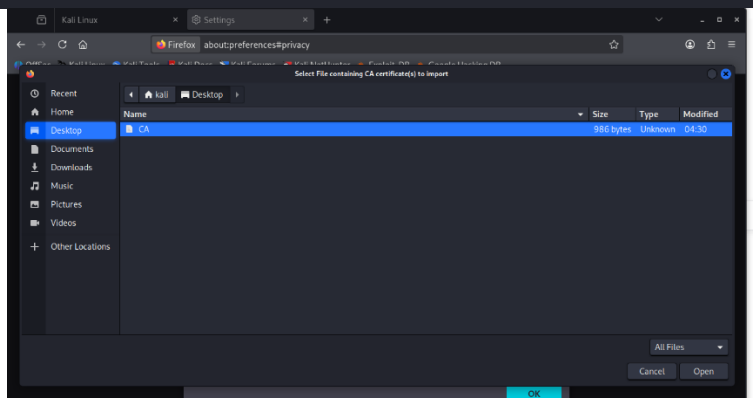


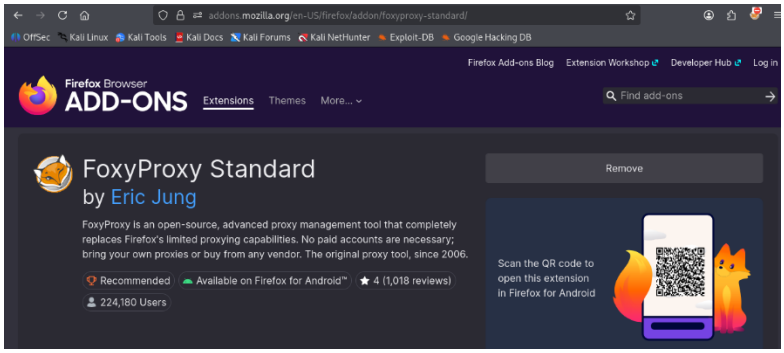


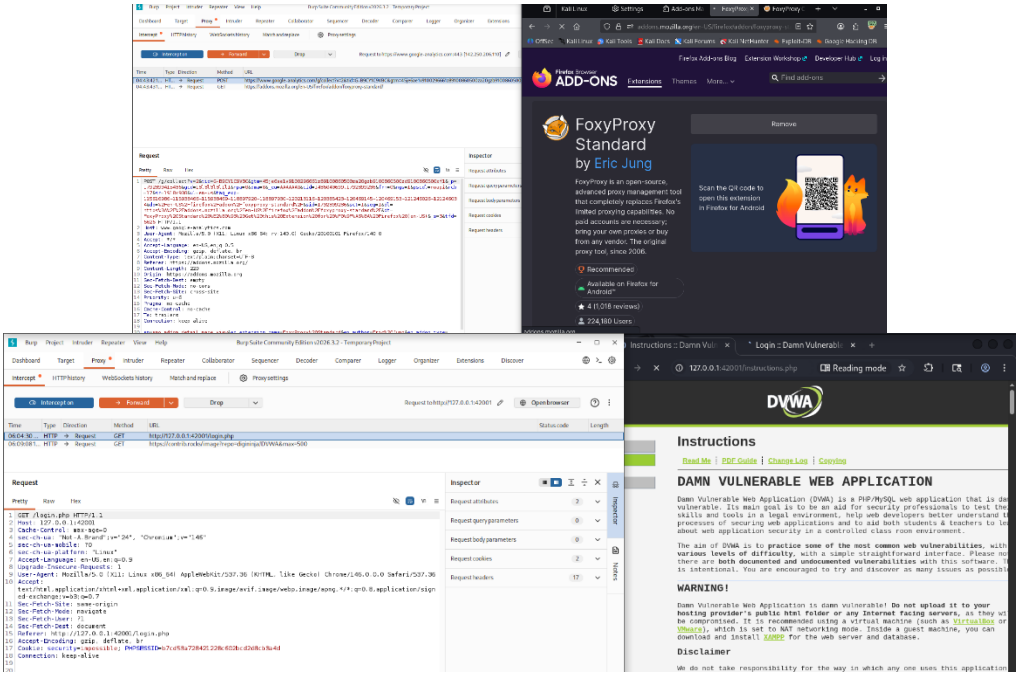



Firefox about:preferences#privacy
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Your browser is being managed by your organization.
Find in Settings
General
Home
Search
Privacy & Security
Sync
More from Mozilla
Deceptive Content and Dangerous Software Protection
Block dangerous and deceptive content Learn more
Block dangerous downloads
Warn you about unwanted and uncommon software
Certificates
Query OCSP responder servers to confirm the current validity of certificates
View Certificates...
Security Devices...
HTTPS-Only Mode
Only allows secure connections to websites. Firefox will ask before connecting insecurely.
How HTTPS-Only works
Enable HTTPS-Only Mode in all windows
Enable HTTPS-Only Mode in private windows only
Manage Exceptions...


Downloading Certificate
You have been asked to trust a new Certificate Authority (CA).
Do you want to trust "PortSwigger CA" for the following purposes?
Trust this CA to identify websites.
Trust this CA to identify email users.
Before trusting this CA for any purpose, you should examine its certificate and its policy and procedures (if available).
View
Examine CA certificate
Cancel
OK


Kali Linux
Settings
Firefox about:preferences#privacy
Select File containing CA certificate(s) to import
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Other Locations
kali
Desktop
Name
Size
Type
Modified
CA
986 bytes
Unknown
04:30
All Files
Cancel
Open
OK


addons.mozilla.org/en-US/firefox/addon/foxyproxy-standard/
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Firefox Add-ons Blog
Extension Workshop
Developer Hub
Log in
Firefox Browser ADD-ONS
Extensions
Themes
More...
Find add-ons
FoxyProxy Standard
by Eric Jung
FoxyProxy is an open-source, advanced proxy management tool that completely replaces Firefox's limited proxying capabilities. No paid accounts are necessary; bring your own proxies or buy from any vendor. The original proxy tool, since 2006.
Recommended
Available on Firefox for Android™
4 (1,018 reviews)
224,180 Users
Remove
Scan the QR code to open this extension in Firefox for Android



Task 23: Reconnaissance and Web Application Mapping

Objective: To explore the web application and identify its endpoints, login forms, input fields, parameters, and APIs using Burp Suite.

Steps Performed

1. Opened Burp Suite and configured the browser to use the Burp proxy.
2. Opened the authorized practice website and browsed its pages.
3. Navigated to Target → Site map to view the discovered endpoints.
4. Identified login forms, search fields, and other user input fields.
5. Checked GET and POST requests and noted their parameters.
6. Recorded the discovered endpoints for further vulnerability testing.

Endpoint Mapping Table

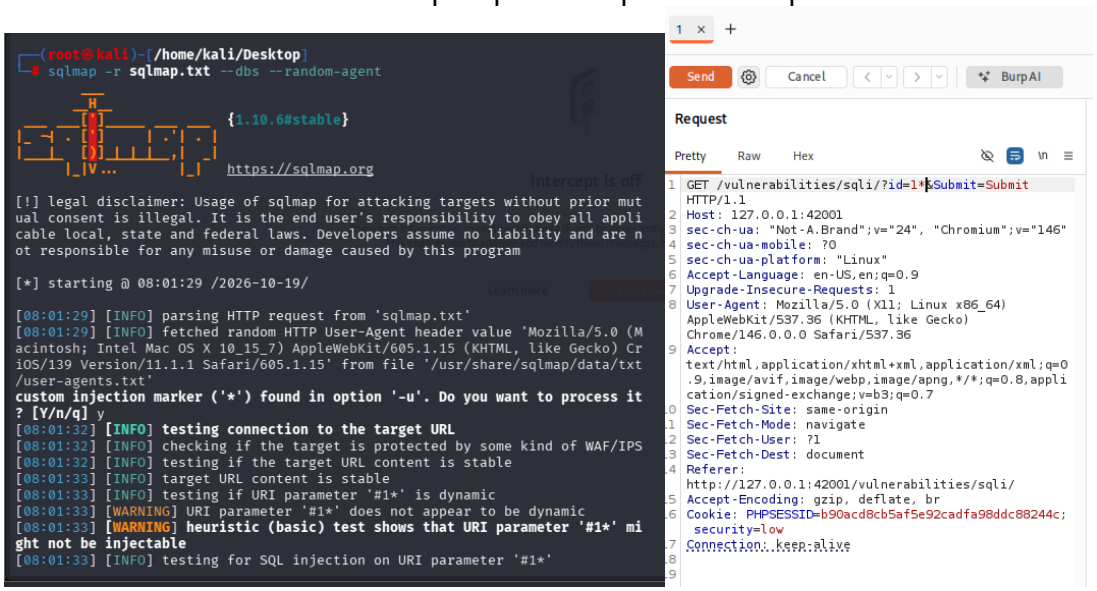
Endpoint	Method	Purpose	Testing Focus
/login.php	GET/POST	User login	Authentication
/setup.php	GET/POST	Application setup	Access control
/instructions.php	GET	Instructions page	Access control
/instructions.php?doc=...	GET	Document-related request	Parameter validation

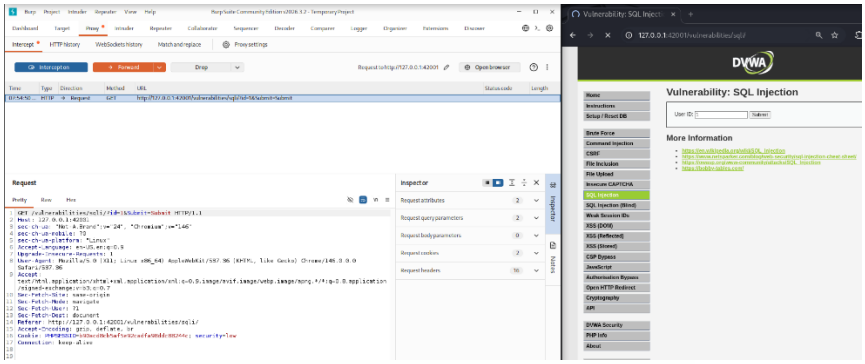
Task 24: Vulnerability Testing Using Burp Suite

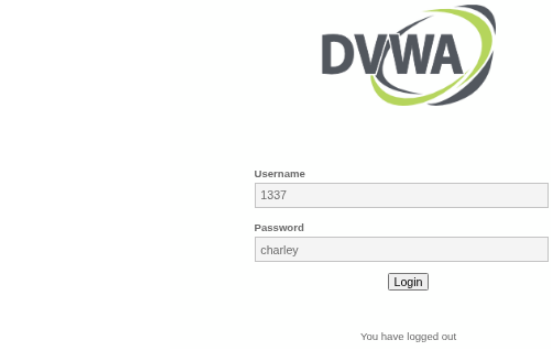
The purpose of this task was to test common web application vulnerabilities using Burp Suite Community Edition in an authorized practice environment.

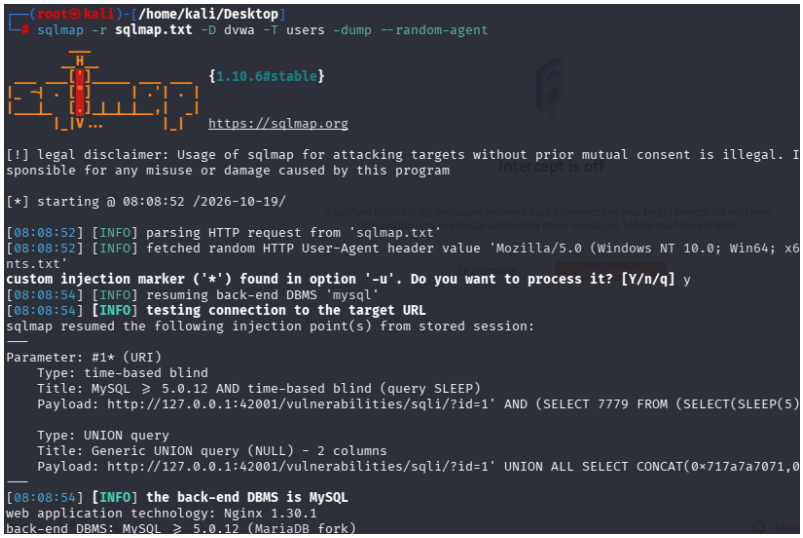
1. SQL Injection (SQLi)

- Step 1: Open the practice website and identify a login or input field.
- Step 2: Capture the request using Burp Proxy and send it to Repeater.
- Step 3: Test a payload such as ' OR 1=1 -- in the authorized lab.
- Observed Response: Check for database errors or unexpected changes in the response.
- Result: Record SQL Injection as confirmed only if the behavior is reproducible and supported by evidence.
    - Screenshot: Burp Repeater request and response.



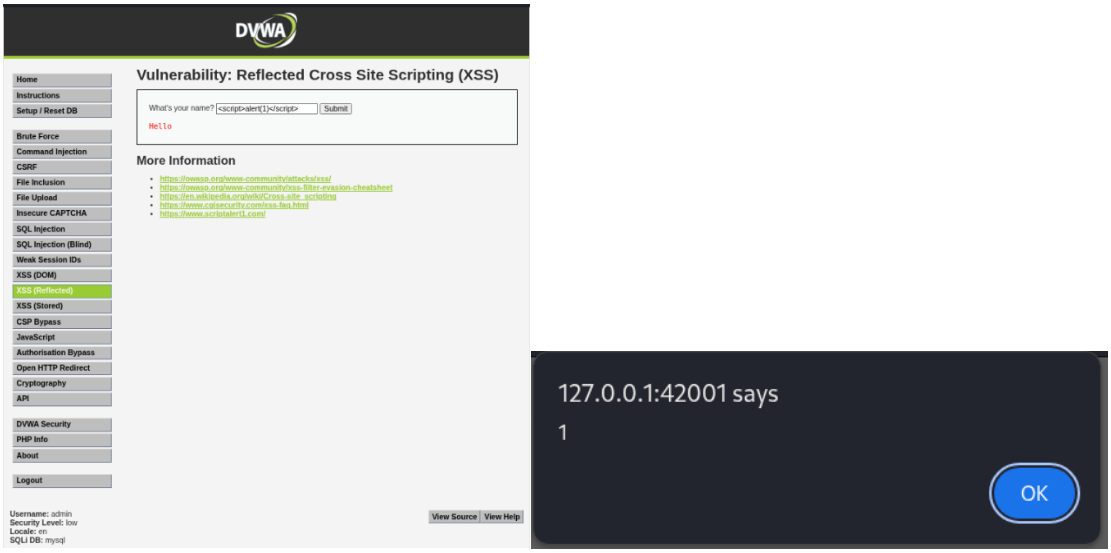






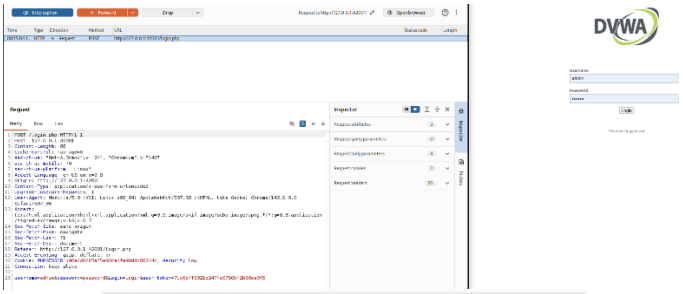
2. Cross-Site Scripting (XSS)

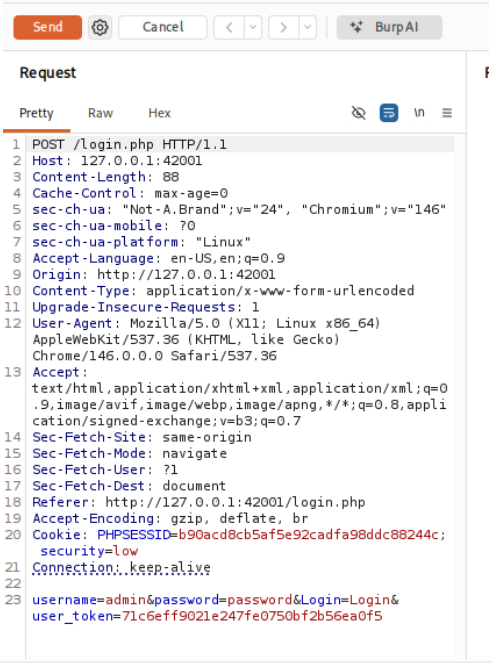
- Step 1: Identify a search box or other input field.
- Step 2: Capture the request in Burp Suite.
- Step 3: Test <script>alert(1)</script> in the practice application.
- Observed Response: Check whether the input is safely displayed or executes as JavaScript in the browser.
- Result: Confirm XSS only if script execution is demonstrated.
- Screenshot: Test input and browser result.

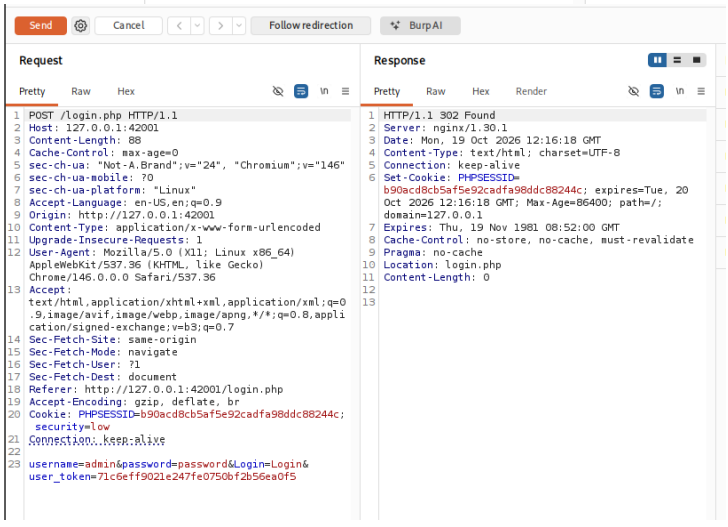


3. Authentication Testing

- Step 1: Open the login page and use a test account.
- Step 2: Capture the login request using Burp Proxy.
- Step 3: Check how the application handles invalid credentials and access to protected pages.
- Observed Response: Record whether login is rejected correctly and unauthorized access is prevented.
- Result: Report a vulnerability only if an authentication weakness is demonstrated.
- Screenshot: Login request and server response, with passwords and session tokens hidden.

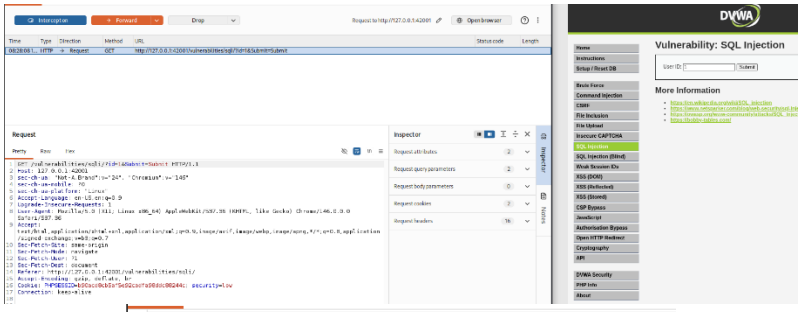




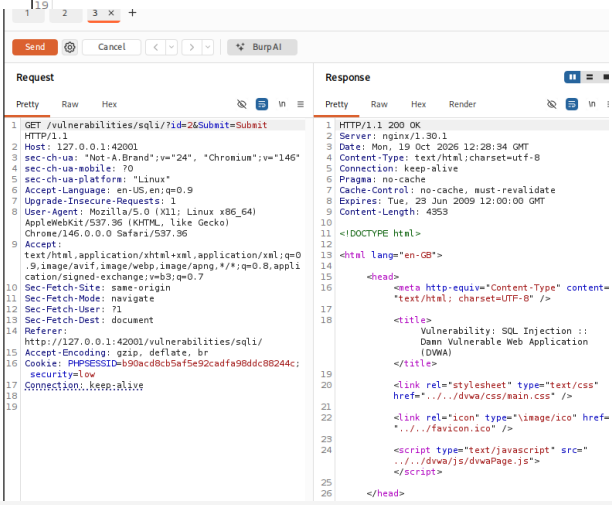


4. Parameter Tampering

- Step 1: Identify a request containing a parameter, such as quantity=1.
- Step 2: Send the request to Burp Repeater.
- Step 3: Modify the parameter to another value and resend the request.
- Observed Response: Check whether the server validates the modified value and enforces the application's rules.
    - Result: Confirm parameter tampering as a vulnerability only if the modification causes an unauthorized or unexpected action.



```
GET /vulnerabilities/sqli/?id=2&Submit=Submit HTTP/1.1
Host: 127.0.0.1:42001
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecl
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://127.0.0.1:42001/vulnerabilities/sqli/
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=b90acd8cb5af5e92cadfa98ddc88244c; security=low
Connection: keep-alive
```



Request

```
GET /vulnerabilities/sqli/?id=2&Submit=Submit
HTTP/1.1
Host: 127.0.0.1:42001
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0
.9,image/avif,image/webp,image/apng,*/*;q=0.8,appli
cation/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer:
http://127.0.0.1:42001/vulnerabilities/sqli/
Accept-Encoding: gzip, deflate, br
Cookie: PHPSESSID=b90acd8cb5af5e92cadfa98ddc88244c;
 security=low
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
Server: nginx/1.30.1
Date: Mon, 19 Oct 2026 12:28:34 GMT
Content-Type: text/html;charset=utf-8
Connection: keep-alive
Pragma: no-cache
Cache-Control: no-cache, must-revalidate
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Content-Length: 4353

<!DOCTYPE html>

<html lang="en-GB">

    <head>
        <meta http-equiv="Content-Type" content=
        "text/html; charset=UTF-8" />

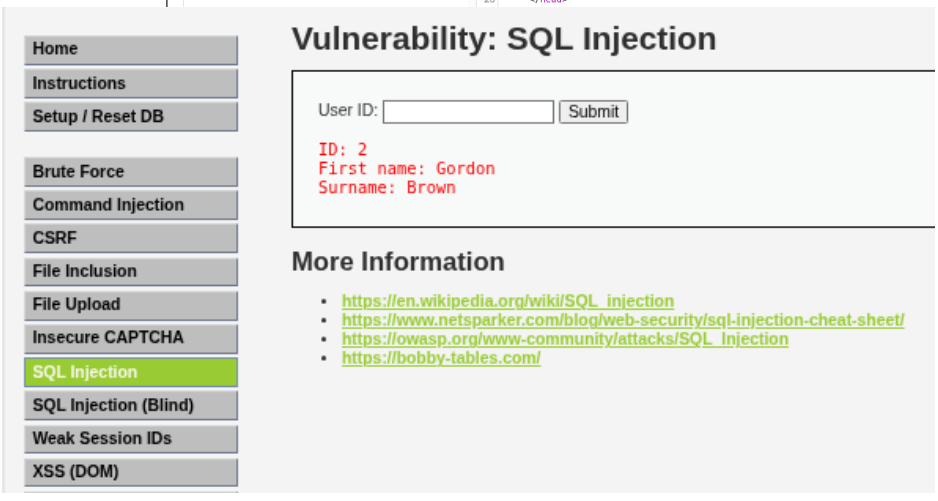
        <title>
            Vulnerability: SQL Injection ::
            Damn Vulnerable Web Application
            (DVWA)
        </title>

        <link rel="stylesheet" type="text/css"
        href="../../dvwa/css/main.css" />

        <link rel="icon" type="\image/ico" href=
        "../../favicon.ico" />

        <script type="text/javascript" src="
        ../../dvwa/js/dvwaPage.js">
        </script>

    </head>
```



Task 25: Directory and Endpoint Discovery

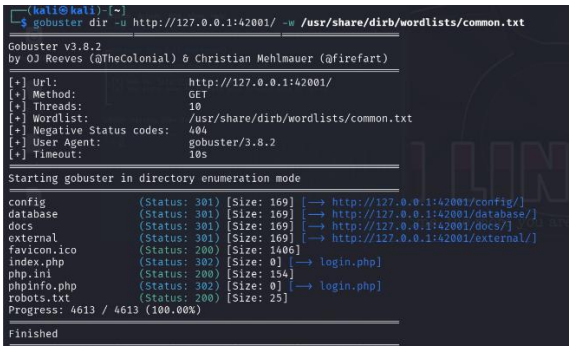
Objective: To discover hidden directories and endpoints in an authorized practice web application.

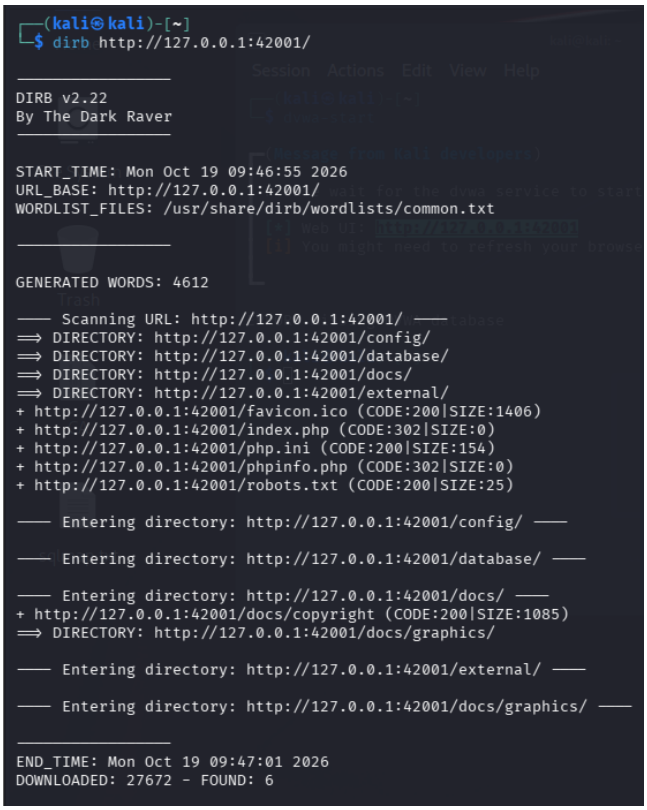
Steps Performed:

1. Opened the practice website in Burp Suite and browsed its available pages.
2. Used Target → Site map to identify discovered URLs and endpoints.
3. Alternatively, used Gobuster to search for directories using a wordlist.

   Gobuster Command:

   gobuster dir -u http://127.0.0.1/ -w /path/to/wordlist.txt
4. Identified possible paths such as /admin, /login, and /api (examples only).
5. Checked the discovered paths and recorded their HTTP status codes.
6. Captured a screenshot of the results for the report.





Task 26 – Vulnerability Findings

Use the following format for each vulnerability in your Week 7 VAPT report. The steps are short and easy to understand.

1. SQL Injection

Finding ID: VAPT-01

1. Vulnerability Name: SQL Injection
2. Severity: Medium (based on lab evidence)
3. Affected URL: Actual login or input endpoint tested.
4. Description: SQL Injection occurs when user input improperly affects database queries.
5. Steps to Reproduce:
    1. Open the authorized practice application.
    2. Capture the input request using Burp Proxy.
    3. Send the request to Repeater.
    4. Enter a SQL Injection test payload.
    5. Analyze the response for evidence of SQL query manipulation.
6. Proof of Concept (PoC): Attach a Burp request/response screenshot.
7. Impact: An attacker may access or modify database information.
8. Remediation: Use parameterized queries and validate input.
9. Result: Confirmed / Not reproduced / Inconclusive.

2. Cross-Site Scripting (XSS)

Finding ID: VAPT-02

1. Vulnerability Name: Cross-Site Scripting (XSS)
2. Severity: Medium (based on lab evidence)
3. Affected URL: Actual search or input endpoint tested.
4. Description: XSS occurs when untrusted input executes as JavaScript in a user's browser.
5. Steps to Reproduce:
    1. Open the practice application.
    2. Enter a test payload in an input field.
    3. Capture the request in Burp Proxy.
    4. Inspect the response and browser behavior.

5. Verify whether the input executes as a script.

6. Proof of Concept (PoC): Attach a screenshot of the test and confirmed browser behavior.
7. Impact: An attacker may perform actions in a victim's browser or access exposed page data.
8. Remediation: Apply context-aware output encoding and safe input handling.
9. Result: Confirmed / Not reproduced / Inconclusive.

3. Broken Access Control

Finding ID: VAPT-03

1. Vulnerability Name: Broken Access Control
2. Severity: Medium (based on lab evidence)
3. Affected URL: Actual protected endpoint tested.
4. Description: This occurs when an application allows users to access resources without proper authorization.
5. Steps to Reproduce:
    1. Log in using an authorized test account.
    2. Capture a request to a protected resource.
    3. Send the request to Burp Repeater.
    4. Test access using the lab's permitted test accounts.
    5. Check whether unauthorized access is possible.
6. Proof of Concept (PoC): Attach the relevant request/response screenshot, hiding session tokens.
7. Impact: Users may access or modify data they are not permitted to access.
8. Remediation: Enforce server-side authorization checks for every protected resource.
9. Result: Confirmed / Not reproduced / Inconclusive.

Important: The severity ratings above are examples. Choose the final severity based on the actual evidence and impact. Mark a vulnerability as Confirmed only when your test demonstrates it successfully.

14. Security Recommendations

1. Prevent SQL Injection: Use parameterized queries to protect database operations.
2. Prevent XSS: Encode output properly before displaying user input.
3. Secure Authentication: Enforce login and access permissions on the server.
4. Validate Input: Check all user inputs on the server side.
5. Use HTTPS: Encrypt communication and secure session cookies.
6. Update Software: Regularly update applications and dependencies.
7. Retest Vulnerabilities: Test again after applying security fixes.

15. Conclusion

During this Week 7 VAPT task, I learned how to use Burp Suite to intercept and analyze web requests, map application endpoints, and test common web vulnerabilities. I also learned to examine server responses, collect screenshots as evidence, and prepare a professional vulnerability report with impact and remediation steps. This practical experience improved my understanding of web application security and vulnerability assessment.

References

1. PortSwigger Web Security Academy
2. Burp Suite Documentation
3. OWASP Web Security Testing Guide
4. DVWA – Damn Vulnerable Web Application
5. OWASP Juice Shop